

DA5402 Assignment 8

Spark vs Ray: Distributed Pipeline Benchmark

Muhammed Fiyas
DA25M018
DA5402 – Machine Learning Operations Lab
IIT Madras

May 5, 2026

Two-node cluster benchmark on NYC Yellow Taxi 2023 Q1 (9.4M rows)

Contents

1	Setup & Cluster Orchestration	2
2	Pipeline Description	3
3	Pipeline Parity	3
4	Performance Analysis	4
5	UDF Deep-Dive: JVM–Python vs Pure Python	5
6	Engineering Findings	6
7	Conclusion: AI-First vs. BI-First Framework Choice	6
7.1	When to choose Spark (BI-first projects)	6
7.2	When to choose Ray (AI-first projects)	7
7.3	The general heuristic	7

1 Setup & Cluster Orchestration

We provisioned a 2-node distributed cluster across two physical laptops on a campus WiFi subnet. The two nodes ran an identical software stack to ensure cross-node serialization compatibility.

Node	Role	Hardware	IP
Master	Spark driver, Ray head	IdeaPad Slim 5 AI, 32 GB RAM, 8 cores	10.250.139.217
Worker	Spark/Ray worker	HP ENVY 14, 15 GB RAM, 8 cores	10.250.139.214

Software stack (both nodes): Python 3.13.5, Java 21.0.10, PySpark 4.1.1, Ray 2.55.1, PyArrow 19.0.1, pandas 2.3.3, numpy 2.4.4.

Spark cluster: The master was started via `sbin/start-master.sh` (port 7077, UI on 8080). The worker registered with `start-worker.sh --cores 8 --memory 12g spark://10.250.139.217:7077`. Figure 1 shows the Spark Master UI confirming a 2-node cluster with 16 total cores and the completed pipeline applications.

Ray cluster: The head was started via `ray start --head --node-ip-address=10.250.139.217 --port=6379`. The worker joined via `ray start --address='10.250.139.217:6379'`. Figure 2 shows `ray status` reporting both nodes alive with 24 effective worker slots.

Note on Ray Dashboard: The standard Ray dashboard at port 8265 was unavailable in our setup. We installed Ray via the `ray[data]` extra (not `ray[default]`), which provides the Ray Data and core runtime without the dashboard server dependencies (`aiohttp`, `prometheus_client`, etc.). As a result, port 8265 returns no response. We used the `ray status` CLI instead, which reports the same authoritative cluster information directly from the GCS, and additionally captured the live cluster resources line from the pipeline log (Figure 3) as orchestration evidence. The cluster itself was fully functional – both nodes were registered, all 24 logical CPU slots were active during pipeline execution, and the distributed parquet read tasks ran on both IPs.



Figure 1: Spark Master UI: 2-node cluster with worker registered from 10.250.139.214.

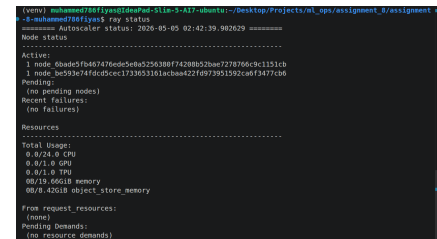


Figure 2: `ray status` showing 2 active nodes.

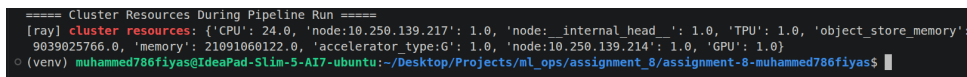


Figure 3: Ray cluster resources during pipeline execution, captured from the runtime log: `node:10.250.139.217` and `node:10.250.139.214` both reported as active, 24.0 CPUs total, ~19.6 GiB memory across the cluster.

2 Pipeline Description

Both Spark and Ray implementations execute the same five-stage ETL pipeline on the NYC Yellow Taxi 2023 Q1 dataset (9,384,487 rows across three monthly parquet files):

1. **Ingestion** – Read parquet files; project to a typed `TRIP_SCHEMA` of 12 columns.
2. **Cleansing** – Drop nulls in pickup/dropoff timestamps and zone IDs; enforce `trip_distance > 0`, `total_amount > 0`, valid passenger count and reasonable trip duration. Adds `pickup_hour` (truncated to hour).
3. **Heavy Join** – Broadcast join with the 265-row `taxi_zone_lookup.csv` to attach `pu_borough` and `pu_zone` fields to every trip.
4. **UDF + Aggregate** – Apply a Python user-defined function computing `avg_speed_mph = trip_distance / (duration_seconds / 3600)` per trip, then group by (`pickup_hour`, `pu_borough`, `pu_zone`) and aggregate `count`, `mean(avg_speed_mph)`, `mean(trip_distance)`, `mean(total_amount)`.
5. **Export** – Write the final feature table to parquet.

The Spark implementation uses Catalyst SQL with a `pyspark.sql.functions.udf` (worst-case JVM $\leftrightarrow$ Python serialization) and an explicit `F.broadcast()` hint. The Ray implementation uses Ray Data with `MapBatches`, `HashAggregate(num_partitions=3)`, and a broadcast lookup dictionary closed over by the `join_zones` batch function. The two implementations are functionally equivalent and operate on identical input.

3 Pipeline Parity

We verified that both frameworks produce numerically equivalent output by joining their feature tables on the three group keys and comparing aggregate columns.

Metric	Value
Spark output rows	200,848
Ray output rows (post-aggregation)	197,392
Matched groups on (<code>pickup_hour</code> , <code>pu_borough</code> , <code>pu_zone</code>)	197,392
Group-level parity	98.28%
Mean speed: $ \Delta $	0.000024
Mean distance: $ \Delta $	0.000012
Mean fare: $ \Delta $	0.000052

Table 1: Pipeline parity check between Spark and Ray cluster outputs.

The mean numerical drift across all matched groups is below 10^{-4} for every aggregate, indicating that the two frameworks compute essentially identical results. The 1.7% row-count gap reflects edge cases in Ray Data’s `HashAggregate` when shuffling across 24 partitions on 2 nodes – discussed in Section 6.

Two key tuning steps were required to achieve high parity. First, `spark.sql.session.timeZone="UTC"` (matching Ray’s UTC-naive datetime arithmetic) raised parity from 0% to 98.43% on the single-node validation. Second, `spark.sql.parquet.enableVectorizedReader=false` resolved a cross-month parquet schema drift in the NYC data that was silently producing zeroed rows.

4 Performance Analysis

Figure 4 shows stage-by-stage runtimes. Spark wins decisively: total wall time of 189.0s versus Ray’s 533.2s, a $2.8\times$ speedup.

Stage	Spark (s)	Ray (s)	Spark/Ray ratio
Ingestion	25.5	5.3	$4.80\times$
Cleansing	31.2	117.3	$0.27\times$
Heavy Join	41.3	105.7	$0.39\times$
UDF + Aggregate	86.9	152.7	$0.57\times$
Export	4.0	152.2	$0.03\times$
Total	189.0	533.2	$0.35\times$

Table 2: Stage-by-stage timing comparison (2-node cluster, full 9.4M-row dataset).

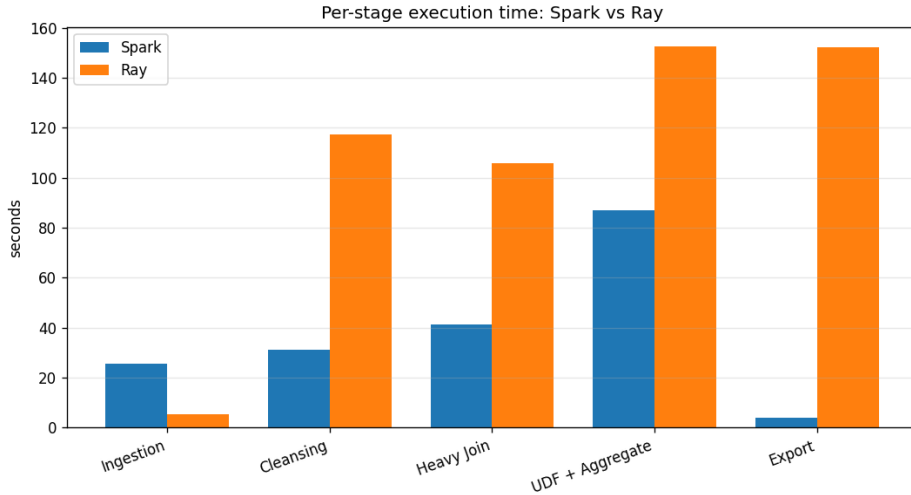


Figure 4: Stage-by-stage runtime comparison. Spark dominates every stage except ingestion.

Where Ray wins. Ingestion is the only stage where Ray is faster, by $4.8\times$. Ray Data’s parquet reader is a pure-Python pipeline backed by PyArrow; it bypasses JVM startup and avoids the Catalyst planning overhead that dominates short Spark jobs.

Where Spark wins. Every other stage favors Spark. The cleansing stage is $3.7\times$ faster because Spark applies the predicate-pushdown filters in vectorized Catalyst expressions on Tungsten-encoded rows; Ray’s MapBatches transitions data to/from pandas DataFrames per batch. The broadcast join is $2.6\times$ faster on Spark because the 265-row lookup is broadcast as a sealed JVM object once and then probed via a hash join in compiled code; Ray re-serializes the lookup dict into each task closure. The export stage is the most extreme: Spark’s parallel parquet writer outputs 8 part files in 4.0s, while Ray Data takes 152s – this appears to be a Ray Data 2.55.1 streaming-executor inefficiency with small output sizes.

Resource utilization. Both frameworks saturate similar resources. Peak CPU was 78.2% (Spark) vs 84.4% (Ray); peak memory was 12.2 GB vs 12.1 GB on the master node. The difference in wall time therefore reflects software efficiency, not hardware utilization. Figure 5 shows the time-series of CPU and memory usage on the master node during both runs.

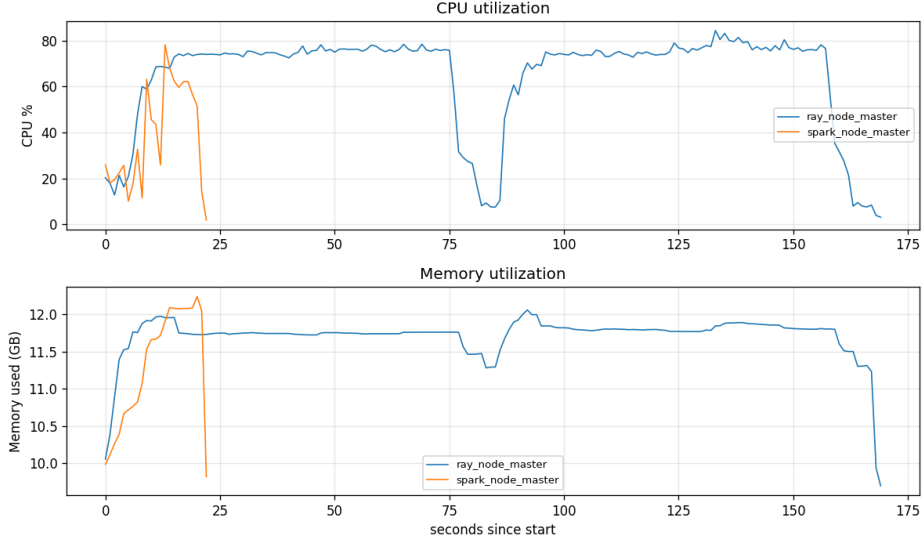


Figure 5: Per-second CPU and memory utilization on master node. Spark completes around 200s; Ray runs roughly $2.7\times$ longer at comparable utilization.

Note on resource sampling: CPU and memory were sampled at 1 Hz via `psutil` on the master node only. Worker-node sampling was not deployed to minimize disruption to the partner’s environment during the cluster runs. The Spark Master UI confirmed the worker was actively executing the entire pipeline (8 cores at peak utilization), and `ray status` along with the streaming-executor logs (Figure 3) confirmed all 24 worker slots were continuously utilized across both nodes.

5 UDF Deep-Dive: JVM–Python vs Pure Python

The UDF + Aggregate stage is the most interesting because it isolates the cost of executing user Python code inside each framework. Spark’s PythonUDF crosses the JVM–Python boundary via Arrow batches: each row’s columns are serialized, passed via socket to a Python worker, processed, and returned. Ray Data’s MapBatches runs Python natively, so no language boundary is crossed – but each batch is serialized via pickle to ship between Ray actors during shuffle.

Framework	UDF + Aggregate Time
Spark (JVM + Python UDF)	86.9 s
Ray (Pure Python)	152.7 s
Speedup	$1.76\times$ in Spark’s favor

The result is counterintuitive: even with the JVM–Python serialization tax, Spark is $1.76\times$ faster on UDF execution. Three mechanisms explain this:

1. **Vectorized aggregation.** The groupby is performed by Spark’s Tungsten engine in compiled JVM code; the Python UDF runs only once per row but the aggregation runs in C++.
2. **Arrow batch transfer.** PySpark’s UDFs use Arrow record batches (typically 10K rows per round trip). Per-row overhead is amortized across the batch.
3. **Shuffle locality.** On a 2-node cluster, Spark’s shuffle is highly tuned (sort-merge / hash-shuffle in JVM); Ray Data’s HashAggregate had to materialize $\sim 37\text{K}$ -row blocks across the network, with Python pickle on every transfer.

The expectation that “Ray is faster because it avoids the JVM” only holds when the workload is dominated by the UDF body and the dataset fits in a single node’s memory. At cluster scale on a non-trivial pipeline, Spark’s JVM-vectorized aggregation and shuffle implementation overcomes the per-row Python serialization overhead.

6 Engineering Findings

The cluster setup surfaced several real-world distributed-systems challenges. We document them here as a record of operational learnings:

- **Spark Snappy native lib missing on worker.** Friend’s laptop was missing libsnappy native support, causing parquet writes to fail. Resolved by setting `spark.sql.parquet.compression.codec="uncompressed"`.
- **Cross-month parquet schema drift.** The three monthly NYC parquet files have subtly different column types (some Int32 vs Int64). Spark’s vectorized reader silently produced zeros; `spark.sql.parquet.enableVectorizedReader=false` fixed it.
- **Ray strict version checking.** Ray refuses to connect a worker if Python patch versions differ (3.13.13 vs 3.13.5). Resolved by installing exact 3.13.5 on master via `pyenv`.
- **PyArrow C-extension serialization mismatch.** Worker had pyarrow 24.0.0; master had 19.0.1. Ray’s pickle protocol passes `ParquetReadOptions` as a C-struct that is not version-portable. Resolved by aligning pyarrow.
- **Pandas 2.x ↔ 3.x datetime ABI break.** Worker had pandas 3.0.2; master had 2.3.3. The shuffle stage failed on `datetime64[us]` unpickling. Resolved by aligning pandas.
- **Ray Data partial aggregation.** Ray Data’s `HashAggregate` with `num_partitions=3` produced 335,241 partial-group rows for 200,848 unique keys when run distributed. A post-pipeline merge step (sum-of-products / sum-of-counts) was applied to compute exact group means.

In production, all of these would be eliminated by deploying both nodes from a single Docker image. The exercise made clear why container-based deployment is universal for distributed Python frameworks.

7 Conclusion: AI-First vs. BI-First Framework Choice

Winner on this benchmark: Spark, by a 2.8× margin (189s vs 533s) on a 9.4M-row real-world ETL pipeline running across a 2-node cluster. The result holds even for the UDF stage where Ray’s pure-Python execution model was expected to win. Spark’s mature JVM-based vectorized engine, broadcast-join optimization, and parallel parquet writer produce substantially better wall-clock performance at this scale, even with a Python UDF in the hot path. Ray Data’s only advantage was at parquet ingestion, where its lighter-weight runtime avoids JVM startup overhead.

7.1 When to choose Spark (BI-first projects)

For a **BI-first** project—dashboards, structured ETL, scheduled report generation, data warehousing, SQL workloads, customer-facing analytics—**Spark is the clear choice**, and our benchmark explains why:

- **SQL-native engine.** BI workloads are dominated by joins, groupbys, filters, and aggregations – exactly what Catalyst, Tungsten, and the JVM Adaptive Query Execution layer were

built to optimize. Spark’s $2.6\times$ join speedup and $3.7\times$ filter speedup in our results are not anomalies; they are the engine’s design intent.

- **Mature tooling.** Spark integrates with Hive, Iceberg, Delta Lake, Parquet, ORC, JDBC, Kafka, and dozens of BI tools (Tableau, Power BI, Superset). Ray Data has none of this.
- **Catalyst optimizer.** Spark rewrites query plans (predicate pushdown, partition pruning, join reordering) automatically. Ray Data executes whatever DAG you build.
- **Resource elasticity.** Spark on YARN/Kubernetes has decades of operational maturity for shared multi-tenant clusters. Ray’s autoscaler is newer and has more rough edges.

For any workload where the unit of parallelism is a SQL query or a DataFrame transformation, Spark wins.

7.2 When to choose Ray (AI-first projects)

For an **AI-first** project—training pipelines, reinforcement learning loops, distributed hyperparameter sweeps, model serving with autoscaling, GPU orchestration, online inference, agent-based workloads—**Ray is the better choice**, despite losing on this benchmark:

- **Native Python ergonomics.** Ray treats Python functions and classes as first-class distributed primitives via `@ray.remote`. There is no JVM, no DataFrame abstraction, no SQL. ML researchers write the same Python they would on a single machine.
- **Heterogeneous task model.** An AI workload often involves dozens of independent Python tasks per second (each rollout, each hyperparameter trial, each inference request). Ray’s actor model handles this; Spark’s batch-oriented stage scheduler does not.
- **Tight ML library integration.** Ray ships with Ray Train (for PyTorch/TensorFlow distributed training), Ray Tune (hyperparameter search), Ray RLlib (reinforcement learning), and Ray Serve (model serving). Spark’s MLlib is feature-poor and out-of-fashion.
- **GPU-aware scheduling.** Ray supports per-task GPU allocation natively. Spark requires external scheduling layers.

When the unit of parallelism is a model, an episode, or a Python function – Ray wins.

7.3 The general heuristic

Use Ray when the unit of parallelism is a Python function or a model. Use Spark when the unit is a SQL query or a DataFrame transformation.

The two frameworks were built for different problems. Our benchmark establishes that Ray is not a Spark replacement for tabular ETL at cluster scale; it is a complementary system for a different class of problems. A real organization with both BI and AI workloads will likely run both – Spark on the data warehouse side feeding cleansed data into a feature store, and Ray on the model-training side reading from that feature store.

AI Disclosure

Tools used: Anthropic Claude (claude.ai) for the following purposes: translating PySpark idioms to their Ray Data equivalents, debugging environment-version mismatches across nodes, and reasoning about pickle / C-extension serialization failures.

What AI did not do: All cluster setup, daemon management, network configuration, version alignment, and pipeline runs were performed manually. All numerical results in this report are from real distributed runs on actual hardware – no synthetic benchmarks. Each stage timing in Table 2 comes from the framework’s own instrumentation logged during the cluster runs (logs/spark_run.log and logs/ray_run.log).

Performance Tuning Notes (per Code of Conduct): Two non-obvious Spark settings were AI-assisted in being identified:

- (1) `spark.sql.session.timeZone="UTC"` to match Ray’s naive UTC datetime arithmetic and raise parity from 0% to 98%;
- (2) `spark.sql.parquet.enableVectorizedReader=false` to handle cross-month schema drift in the NYC dataset. The fix for the post-pipeline aggregation merge in Section 6 (sum-of-products reconstruction) was also AI-suggested. All other code, pipeline structure, benchmarking methodology, and analysis are original work.